MEDIUM-1：位级操作

【任务 1】

舰长手册上留下的密码是：211（十进制） 和 10110110（二进制）。

1. 请写出十进制 211 对应的二进制表示。
2. 请写出二进制 10110110 对应的十进制表示。
3. 在 C/C++ 中，如何快速判断一个十进制整数是奇数还是偶数?

1.11010011

211/2=105（1）105/2=52（1）52/2=26（0）26/2=13（0）13/2=6（1）6/2=3（0）3/2=（1）1/2=0（1）11010011

2.182

2+4+16+32+128=182

3.通过其转换为二进制数的最低位来快速判断，最低位为1，则为奇数，最低位为0，则为偶数。

【任务 2】

为了修复能量阵列，请完成下列题目（将正确的可执行代码提交至**markdown文档**中，并附上运行结果的截图）：

1.阅读并推导以下 C/C++ 代码的输出结果，并简要解释你的计算结果：

```c
#include <stdio.h>

int main() {
    unsigned char a = 12;  // 二进制: 00001100
    unsigned char b = 25;  // 二进制: 00011001

    unsigned char res1 = a & b;
    unsigned char res2 = a | b;
    unsigned char res3 = a ^ b;
    unsigned char res4 = (a << 2) | (b >> 1);

    printf("%d %d %d %d\n", res1, res2, res3, res4);
    return 0;
}
```

结果：8 29 21 60

解释：

res1 = a & b

规则：两个都是 1 才为 1。

逐位对齐：

```
a: 0 0 0 0 1 1 0 0
b: 0 0 0 1 1 0 0 1
&: 0 0 0 0 1 0 0 0
```

结果 $00001000_2 = 8_{10}$。

res2 = a | b

规则：只要有一个 1 就为 1。

```
a: 0 0 0 0 1 1 0 0
b: 0 0 0 1 1 0 0 1
|: 0 0 0 1 1 1 0 1
```

结果 $00011101_2 = 16 + 8 + 4 + 1 = 29_{10}$。

res3 = a ^ b

规则：不同为 1，相同为 0。

```
a: 0 0 0 0 1 1 0 0
b: 0 0 0 1 1 0 0 1
^: 0 0 0 1 0 1 0 1
```

结果 $00010101_2 = 16 + 4 + 1 = 21_{10}$。

res4 = (a << 2) | (b >> 1)

先算 `a << 2`：
`00001100` 左移 2 位变成 $00110000_2 = 48_{10}$。

再算 `b >> 1`：
`00011001` 右移 1 位变成 $00001100_2 = 12_{10}$。

最后按位或：

```
00110000
| 00001100
```

$= 00111100_2 = 32 + 16 + 8 + 4 = 60_{10}$。

所以最终输出是：`8 29 21 60`。

2.给定一个整数x（十进制），同时指定一个位数n，确定这个整数x的二进制表示上第n位是0还是1。例如：251的二进制表示是11111011，n=3，结果返回0。

```
int getBit(int x, int n) {
    return (x >> (n - 1)) & 1;
}
```

3.给定一个整数x（十进制），指定一个位数n，给定一个修正值t（0或者1），将整数x的二进制表示上的第n位改为修正值t。输出被修改后的整数

```
int changeBit(int x, int n, int t) {
    int mask = 1 << (n - 1);          // 00001000
    x = x & ~mask;                    // 把第 n 位清零
    x = x | (t << (n - 1));           // 把第 n 位设为 t
    return x;
}
```

4.给定一个有符号32位整数x，找到其二进制表示上从右开始的第一个1，并输出该数，例如：x=10,二进制表示为1010,第一个1在第二位，所以输出二进制表示为10的数2。

lowbit 问题。核心公式是 `x & (-x)` —— 在补码表示下，`-x = ~x + 1`，二者按位与后恰好只保留最右侧的那个 1。

```
int lowbit(int x) {
    return x & (-x);
}
```

5.现在有两串无符号数a和b的二进制编码，并且有a<b，每个数字的大小在100位以内，现在问区间[a,b]内&的结果。

按位与的特点是：**只要区间内某个位出现过一次 0，这一位最终结果就是 0**（因为 0 和任何数与都是 0）。

从 a 数到 b（a < b），我们从高位往低位看：

- 高位上 a 和 b 一样的地方 → 中间所有数这一位都不会变 → 可以保留
- 一旦遇到第一个 a 和 b 不一样的位 → 说明从这一位开始，数在递增过程中**必然经历过 0 也经历过 1** → 这一位以及后面所有低位，按位与全是 0

结论：

区间 [a, b] 内所有数按位与（&）的结果 = a 和 b 的二进制最长公共前缀，后面全部补 0。

```c
#include <stdio.h>
#include <string.h>

#define MAXLEN 105

int main() {
    char a[MAXLEN], b[MAXLEN];
    scanf("%s %s", a, b);

    int la = strlen(a);
    int lb = strlen(b);
    int maxlen = la > lb ? la : lb;

    // 把短的前面补0，对齐到相同长度
    char aa[MAXLEN] = {0}, bb[MAXLEN] = {0};
    int i;
    for (i = 0; i < maxlen - la; i++) aa[i] = '0';
    strcat(aa, a);
    for (i = 0; i < maxlen - lb; i++) bb[i] = '0';
    strcat(bb, b);

    // 找最长公共前缀
    int prefix_len = 0;
    for (i = 0; i < maxlen; i++) {
        if (aa[i] == bb[i]) prefix_len++;
        else break;
    }

    // 结果：前面公共前缀保留，后面全补0
    char result[MAXLEN] = {0};
    for (i = 0; i < prefix_len; i++) result[i] = aa[i];
    for (i = prefix_len; i < maxlen; i++) result[i] = '0';

    // 去掉前导零（但至少留一位，保证结果是0时输出"0"）
    int start = 0;
    while (start < maxlen - 1 && result[start] == '0') start++;

    printf("%s\n", result + start);
    return 0;
}
```

`strcat` 是**字符串粘贴**—— 把第二个字符串，接到第一个字符串的尾巴后面。

`%s` 打印字符串的规则是：**从你给的位置开始，一个字符一个字符往后打，直到碰到 `\0`（结束符）才停**。

而 `result + start` 就是告诉 `%s`：**别从头开始打了，从第 `start` 个位置开始打**。

思考题：

在接触了C语言这么长时间后，相信你已经对逻辑运算了如指掌了，那么你能说说看逻辑运算和位运算的区别和联系吗。思考并提交于markdown文档。

一、逻辑运算

C 语言里只有三个：&&（与）、||（或）、！（非）。

规则

- 任何非 0 的值都算 **真**，0 算 **假**。
- 结果只有两种：**真** → **1**，**假** → **0**（是 int 类型，不是 bool）。
- 它把整个表达式当成一个"真/假"来处理，**不关心内部每一位长什么样**。

短路求值（重点）

逻辑运算**从左往右算，能确定结果就立刻停，右边不算**。

```
if (a != 0 && 10 / a > 1) { ... }
//    如果 a==0，左边已经是假，右边 10/a 就不会执行
//    否则就除以 0 了—这就是短路的意义
```

```
if (p == NULL || pif (p == NULL || p->x > 0) { ... }
//   p==NULL 为真时，右边 p->x 不会执行，避免空指针->x > 0) { ... }//  ↑ p==NULL 为真时，右边 p->x 不会执行，避免空指针
```

> 口诀：**与遇假即停，或遇真即停。**

二、位运算

C 语言里有六个：& | ^ ~ << >>。

规则

- 直接操作整数的**每一个二进制位**。
- 两个数**对应位分别运算**，结果是一个完整的整数，**不只是 0/1**。
- **没有短路**，两边都一定会被算。

运算符	含义	例子（4位）
&	按位与：两位都为1才为1	0101 & 0011 = 0001
|	按位或：有一个1就为1	0101 | 0011 = 0111
^	按位异或：不同为1，相同为0	0101 ^ 0011 = 0110
~	按位取反：0变1，1变0	~0101 = ...1010（补码）
<<	左移：低位补0，相当于×2	0011 << 1 = 0110
>>	右移：高位补符号位，相当于÷2	0110 >> 1 = 0011

三、核心区别对比表

对比维度	逻辑运算 && | !	位运算 & | ^ ~ << >>
操作单位	整个值（非0即真）	二进制的**每一位**
操作对象个数	双目（! 是单目）	双目（~ 是单目）
结果可能取值	**只有 0 或 1**	任意整数（按位算出来是啥就是啥）
短路求值	✅ 有	❌ 没有，两边都算
用途	if / while 里做条件判断	操作具体 bit、压缩状态、快速乘除
典型场景	if (a>0 && b>0)	n & 1 判奇偶、x & (-x) 取 lowbit

四、最容易搞混的例子

1. & vs &&

```
1 & 2      1 && 2
```

2. | vs ||

```
1 | 2  //=3    1 || 2 // =1
```

3. ~ vs !

在补码中：−x=~x+1

```
!0        // = 1（逻辑非：假变真）
~0        // = -1（位取反）

!5        // = 0（非0即真，取反后为假）
~5        // = -6（按位取反，不是简单变负号）
```

4. **一个数字是几，不影响逻辑运算的真假**

```
5 && 6    // = 1（两个非0，都是真）
0 && 5    // = 0（短路，右边不算）
```

但位运算会认真算每一位：

```
5 & 6     // = 4（0101 & 0110 = 0100）
```

五、联系

1. **都是运算符，都能出现在表达式里**，返回值都是 int。
2. **单目运算符** ! 和 ~ 都是从右往左结合（!!a、~~a）。
3. **在 if 里效果一样，但语义不同**：

```
if (a & 1)     // 判断 a 的最低位是不是 1（位运算）
if (a != 0)    // 判断 a 是不是非0（逻辑判断）
// 这两个在 a 是 1、3、5... 时结果一样，但意思不同
```

六、位运算的常用技巧

结合之前的题：

目的	写法	说明
判断奇偶	n & 1	最低位为1就是奇数
取最低位的1（lowbit）	n & (-n)	你做过的那道题
交换两个数	a ^= b; b ^= a; a ^= b;	不用临时变量
乘2 / 除2	n << 1 / n >> 1	比乘除法快
把第 k 位设为1	x | (1 << k)	开灯
把第 k 位清为0	x & ~(1 << k)	灭灯
翻转第 k 位	x ^ (1 << k)	闪灯
判断第 k 位是不是1	(x >> k) & 1	看灯

位运算的定义很简单，但是再简单的定义也可能产生一些有用的性质（比如运算律）。查阅资料了解位运算有哪些性质。

律	公式
交换律	`a & b = b & a`；`a | b = b | a`；`a ^ b = b ^ a`
结合律	`(a & b) & c = a & (b & c)`；`(a | b) | c = a | (b | c)`；`(a ^ b) ^ c = a ^ (b ^ c)`
等幂律	`a & a = a`；`a | a = a`（注意：`a ^ a = 0`，**不**满足等幂）

运算	与 0 作用	与自身作用	与取反作用
`&`	`a & 0 = 0`（零律）	`a & a = a`	`a & ~a = 0`（互补律）
`|`	`a | 0 = a`（恒等律）	`a | a = a`	`a | ~a = 全 1`（互补律）
`^`	`a ^ 0 = a`（恒等律）	`a ^ a = 0`（归零律）	—

`a ^ a = 0` 是异或最常用的性质 —— 可以用来**清零一个数、交换两个变量**（`a ^= b; b ^= a; a ^= b;`）。

分配律:

`&` **可以 "穿透"** `|`**、** `^`，但 `^` 不能穿透 `|`。这是异或和普通加法的关键区别。

德摩根定律:

"与的取反等于取反的或，或的取反等于取反的与"

```
~(a & b) = ~a | ~b
~(a | b) = ~a & ~b
```

吸收率：

`a` 已经包含了的信息，再和 `b` 组合不会改变 `a` 本身。

```
a & (a | b) = a
a | (a & b) = a
```

移位运算的性质：

性质	公式（无溢出时）
左移 = 乘 2 的幂	`a << n = a × 2ⁿ`
右移 = 除 2 的幂（向下取整）	`a >> n = a ÷ 2ⁿ`
移位与位掩码组合	`(a >> n) & mask` 可提取第 n 位开始的若干位

七、异或的特殊性质

1.**自反性 / 还原性**：

`a ^ b ^ b = a`

同一个数连续异或两次就还原，这是异或用于**加密、校验、成对出现**场景的数学基础。

2.**无进位加法**：

`a ^ b` 正好是 `a + b` 去掉进位后的结果；而 `(a & b) << 1` 正好是进位部分，进位要加到高位，所以左移 1 位 `<<1` （等价乘以 2）。

- 所以：`a + b = (a ^ b) + 2 * (a & b)`，这也是**不用加号实现加法**（位运算加法）的原理。进位要加到高位，所以左移 1 位 `<<1` （等价乘以 2）

移位运算中，右移有两种，其中算数右移对应有符号数，逻辑右移对应无符号数，那么为什么会有这种区别呢。查阅资料理解整数的补码表示后回答于markdown文档。

	补什么	用于	效果
逻辑右移	永远补 0	无符号数（`unsigned`）	纯位操作，不维护符号
算术右移	补符号位（0 或 1）	有符号数（`int`）	保持符号，实现 `floor(x/2)`

> 左移之所以不分有符号 / 无符号，是因为左移在低位补 0，对两种解释都是乘 2（溢出除外），不存在符号维护问题。

【任务 3】编程题：字符串交集检测

这题只用讨论两个给定字符串是否出现重复字母，不用计算重复次数。所以，初步思路是各自创建一个int mask，从第0位开始记录每一个字母出现与否，最终通过两个掩码按位与，当且仅当该字母**在两个串里都出现过**某位才为 1 。

要求空间复杂度严格为 O(1)，时间复杂度为 O(N)。

- 时间：两个 `while` 各遍历一次字符串，总次数 = `len(s1) + len(s2)`，所以 `O(N)`。
- 空间：只用了 `mask1`、`mask2` 两个 `int`，没有随字符串长度增长的额外内存，所以 `O(1)`。

```c
#include <stdio.h>

// 请补全以下代码
int hasCommonChar(const char *s1, const char *s2) {
    int mask1 = 0;
    int mask2 = 0;
    while(*s1 != '\0'){
        mask1=mask1 | (1<<(*s1 - 'a'));
        s1++;
    }
     while(*s2 != '\0'){
        mask2=mask2 | (1<<(*s2 - 'a'));
        s2++;
    }
    if(mask1 & mask2)
        return 1;


    return 0;
}
```

【任务 4】给定反应堆的规模 N (1≤N≤15)，请使用位运算实现求解控制棒摆放方案的总数

```
dfs(col, ld, rd):
    if col == all:            // 所有列都放满了
        方案数 + 1
        return

    可用 = (~(col | ld | rd)) & all

    while 可用 != 0:
        p = 可用 & -可用       // 取最低位的空位
        可用 -= p               // 删掉这个空位（准备回溯）

        dfs(col | p, (ld | p) << 1 & all, (rd | p) >> 1 & all)
```

1 核心思想

逐行放置，一行放一个皇后。不用二维数组，只用 **3 个整数** 记录禁放位置。

2 三个状态变量

变量	含义
col	哪些列已经被占用
ld	左斜线投影到当前行的位置
rd	右斜线投影到当前行的位置

> 每一位 = 一列。1 = 这列不能放，0 = 这列能放。

3 每一步怎么算

① 这一行能放哪?

```
int full  = col | ld | rd;      // 三个方向叠起来 = 不能放的位置
int board = (~full) & limit;   // 取反截断 = 能放的位置
```

> & limit 是为了截断高位，防止取反后的高位垃圾干扰。

② 试每一个空位

```
while(board != 0){
    place = board & (-board);    // 取最低位的空位
    board ^= place;              // 划掉这个空位（准备试下一个）

    // 递归下一行
    queen(col | place,                    // 列：放上新皇后
          (left | place) >> 1 & limit,    // 左斜线：下移一行右移一列
          (right | place) << 1 & limit,   // 右斜线：下移一行左移一列
          limit, add);
}
```

③ 什么时候找到解？

```
if(col == limit){   // 所有列都放满了
    (*add)++;
    return;
}
```

---

4 踩坑记录（Debug 总结）

坑 1：= 和 == 搞混

```
if(col = limit)   // 赋值！不是比较！
if(col == limit)  // 正确
```

后果：每次进来都把 col 赋成 limit，答案永远是 1。

坑 2：函数参数里写了赋值语句

```
// 错误：col = col | place 是赋值，会真的修改 col
queen(col = col | place, left = (left | place) >> 1, ...);

// 正确：col | place 是表达式，只算值不改原变量
queen(col | place, (left | place) >> 1, ...);
```

后果：循环里改了原变量，递归回来后变量不是原来的值，下一次循环试的位置全错。

> **核心原则：函数参数里只写表达式，不要写 = 赋值。**

坑 3：指针传参多写了 &

```
void queen(..., int *add) { ... }

queen(..., &add);   // 错误：add 已经是指针了，&add 变成 int**
queen(..., add);    // 正确
```

坑 4：变量没初始化

```
int col;            // 垃圾值!
queen(col, ...);

int col = 0;        // 正确
queen(col, ...);
```

更好的写法是主函数中不出现col等，直接把0赋进queen函数就行

完整代码

```c
#include <stdio.h>
void queen(int col,int left,int right,int limit,int *add){
    if(col==limit){
        (*add)++;
        return;
    }
    int full=col | left | right ;//1表示已经放
    int board=(~full) & limit;//1表示还能放
    int place=0;

    while(board!=0){
        place=board & (-board);
        board=board^place;//1表示还能放
        queen(col | place, (left | place) >>1 ,(right | place) <<1 ,limit ,add);
    }
}
int main(){
    int n,limit;
    scanf("%d",&n);
    limit = (1<<n)-1;
    int add=0;
    queen(0,0,0,limit,&add );
    printf("%d",add);
    return 0;
}
```

运行结果测试





【任务 5】编程题：广度优先搜索 (BFS) 与位运算寻路

一、BFS 基础概念

1.1 什么是 BFS?

BFS = 广度优先搜索 = **一层一层往外扩着走**。

从起点开始：

- 先走 1 步能到的所有格子
- 再走 2 步能到的所有格子
- 再走 3 步……

第一次碰到终点的时候，就是最短步数。

1.2 BFS 为什么能求最短路径?

因为 BFS 是按"步数从小到大"的顺序遍历的。 你第一次走到某个格子的时候，走的一定是最短路径——因为如果有更短的路径，你早就走到了。

二、核心数据结构

2.1 结构体 Node

一句话：Node 就是一个"包裹"，里面装着位置和步数。

> 不用结构体的话，要开两个数组分别存位置和步数，很麻烦。 用 Node 数组，一个格子里同时装着位置和步数，拿出来就是一整个。

2.2 队列 Queue

入队：

把新节点放进队尾，然后 rear 往后挪一个。

出队：

取出队头的节点，然后 front 往后挪一个。

判断队列空：

```
while(front < rear) { ... }
```

队头还没走到队尾，说明还有人没处理。

2.3 位运算 visited

```
int visited = 0;   // 一个整数，每一位代表一个格子
```

- 第 n 位 = 1 → n 号格子走过了
- visited & (1 << n) → 判断 n 号有没有走过
- visited |= (1 << n) → 标记 n 号走过了

> 不用开 bool 数组，一个 int 的 32 位就够记 32 个格子的访问状态。 这就是位运算状态压缩——用最小的内存存最多的状态。

三、4×4 网格坐标转换

3.1 编号 → 行和列

```
int r = pos / 4;
int c = pos % 4;
```

3.2 行和列 → 编号

```
int npos = nr * 4 + nc;
```

四、踩坑记录（Debug 总结）

坑 1：循环里改了原变量

```
// 错误：每次循环都修改 cur.pos，下一次循环就在错的基础上算
for(int i=0;i<4;i++){
    cur.pos = 4*dr[i] + dc[i] + cur.pos;
}

// 正确：用新变量 npos，每次都从原始 pos 开始算
for(int i=0;i<4;i++){
    int npos = pos + 4*dr[i] + dc[i];
}
```

后果： 第一次循环改了 pos，第二次循环就在改完的 pos 上继续加，全错了。

```
// 错误
queue[rear++] = (Node){npos, ++dist};
visited |= (1 << pos);

// 正确
queue[rear++] = (Node){npos, dist+1};
visited |= (1 << npos);
```

原因：各个循环应该独立，不能让先后顺序影响dist的初始值，所以不能用++

> 这和 N 皇后的 Bug 4 是同一个问题——**循环/回溯里不要改原变量。**

坑 2：左右移动跨行

```
// 错误：只检查 0~15，没检查有没有走到上一行/下一行
if(npos > 15 || npos < 0) continue;

// 正确：拆成行和列来检查越界
if(nr < 0 || nr >= 4 || nc < 0 || nc >= 4) continue;
```

后果： 比如 pos=3（第0行最右边），往右走变成 4（第1行最左边），这不是右边，是下一行。

> 拆成行和列来算，就不会有跨行问题了——行越界了就 continue，列越界了就 continue。

五、总结

BFS通用模板

```
// 1. 定义结构体
typedef struct {
    int x, y;     // 坐标
    int dist;     // 步数
} Node;

// 2. 四个方向
int dx[4] = {-1, 1, 0, 0};
int dy[4] = {0, 0, -1, 1};

// 3. BFS 主函数
int bfs(起点坐标) {
    队列初始化
    visited 标记起点

    while(队列没空) {
        取出队头
        如果是终点 → 返回

        四个方向循环:
            算新坐标
            检查越界
            检查障碍
            检查 visited

            入队 + 标记 visited
    }

    return -1;  // 不可达
}
```

完整代码

```c
#include <stdio.h>
#include <stdbool.h>

// 定义简单队列结构用于 BFS
typedef struct {
    int pos;   // 当前节点编号 (0~15)
    int dist;  // 到达当前节点的最短步数
} Node;

int minStepsToCheese(int walls) {
    int start = 0;
    int target = 15;

    // 如果起点或终点本身是墙，直接不可达
    if ((walls & (1 << start)) || (walls & (1 << target))) {
        return -1;
    }

    // BFS 队列与访问位图
    Node queue[16];
    int front = 0, rear = 0;
    int visited = 0;

    // 起点入队并标记已访问 (请使用位运算)
    queue[rear++] = (Node){start, 0};
    visited |= (1 << start);

    // 上、下、左、右四个方向的节点偏移量
    int dr[4] = {-1, 1, 0, 0};//上下差4
    int dc[4] = {0, 0, -1, 1};//左右差1

    //请在TO DO 和END OF TO DO 行之间补全代码:
    //TO DO
    while(front<rear){
        Node cur = queue[front++];
        int pos = cur.pos;
        int dist = cur.dist;
        if(pos==target){
            return cur.dist;
        }
        int r=pos/4;
        int c=pos%4;
        for(int i=0;i<4;i++){
            int nr = r + dr[i];
            int nc = c + dc[i];
            if(nr < 0 || nr >= 4 || nc < 0 || nc >= 4)
                continue;
```

```
            int npos = nr * 4 + nc;

            if(visited & (1 << npos) ){
                continue;
            }
            if ((walls & (1 << npos)) )
            {
                continue;
            }

            queue[rear++] = (Node){npos,dist+1};
            visited |= (1 << npos);
        }
    }
    //END OF TO DO
    return -1; // 无法到达
}

int main() {
    int walls = (1 << 5) | (1 << 10); // 5号和10号格子是墙
    int steps = minStepsToCheese(walls);
    printf("Minimum steps: %d\n", steps); // 应输出 6
    return 0;
}
```

运行结果测试

